Course Structure

Course Structure	
Course Code	BSC10050
Course Category	PF
Course Title	Microprocessor

Teaching Scheme	Lectures	Tutorials	Laboratory / Practical	Project	Total
Weekly load hours	3		2		5
Credits	3		1		4
Assessment Code	TL3				

Assessment Structure

Type of Course	Assessment Scheme Code	Description L-T-P-J-C	CCA1	MT	CCA2	LCA1	LCA2	LCA3	TE	Total
Theory and Lab Course 3	TL3	3-0-1-0-4	10	25	10	5	5	5	40	100

Course Outcomes

Course Outcomes:

On completion of the course, student will be able to–

1. Understand the block diagram of 8086 Microprocessor
2. Understand the Instruction set of 8086
3. Write an assembly language program
4. Understand the advanced Microprocessor

Course Contents

Course Contents:

Unit 1: 8086 Architecture **10 Hours**

Evolution of Microprocessors and types, 8086 Microprocessor, Salient features, Pin descriptions, Architecture of 8086 - Functional Block diagram, Register organization, Concepts of pipelining (5 stage), Memory segmentation, Physical memory addresses generation., Minimum mode and Maximum Mode of 8086

Unit 2: Instruction Set of 8086 Microprocessor **15 Hours**

Addressing modes, Data transfer instructions, Arithmetic Instructions, Logical Instructions, Bit manipulation instructions, String Operation Instructions, Program control transfer or branching Instructions, Process control Instructions, examples based on Instruction Set

Course Contents

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Unit 3: 8086 Assembly Language Programming **10 Hours**

Introduction to Assembly Programming and Assembler Directive

Assembly program on Addition, Subtraction, Multiplication and Division

Sum of Series

Smallest and Largest numbers from array

Sorting numbers in Ascending and Descending order

Block transfer

String Operations - Length, Reverse, Compare, Concatenation, Copy

Unit 4: Advanced Processor **10 Hours**

Introduction to Pentium Processor, Feature of Pentium Pro Processor Architecture of Pentium Pro Processor,

Software Model of Pentium Pro , Pipelining of Pentium Processor , Classification of Advanced Processors

Course Contents - Lab

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Lab Experiments for Microprocessor

Assembly language program for arithmetic operations

Assembly language program to convert the given decimal number to binary, octal number system

Assembly language program to find the largest and smallest among the given numbers

Assembly language program to arrange the given numbers in ascending and descending order

Assembly language program to find the square of a number

Assembly language program to find sum of an array

Resources

Learning Resources:

Reference Books:

1) Microprocessor & interfacing (programming & hardware) Revised Second Edition By Douglas V. Hall Tata McGraw Hill
2) Microprocessor Architecture By B.Ram
3) Microprocessor Architecture, Interfacing & Programming By Mohammad Raffiqzamman

Supplementary Reading:

1) The Intel Microprocessors : Barry B. Brey- Pearson Education Asia
2) The Pentium Microprocessor : James Antonakos

Content

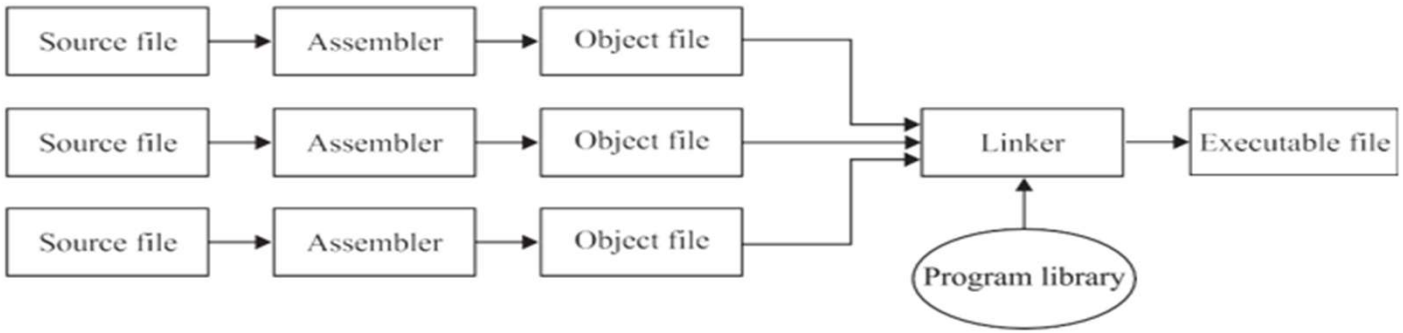
- ✓ Introduction to Assembly Programming
- ✓ Assembler Directive
- ✓ Assembly program on Addition, Subtraction, Multiplication and Division
- ✓ Sum of Series
- ✓ Smallest and Largest numbers from array
- ✓ Sorting numbers in Ascending and Descending order
- ✓ Block transfer
- ✓ String Operations - Length, Reverse, Compare, Concatenation, Copy

Assembly Programming Language

- ✓ Machines can only understand the binary numbers and hence the instructions for computers are written in binary numbers.
- ✓ However, human beings have a great deal of difficulty in understanding and manipulating these binary numbers.
- ✓ So designers develop a way to write the instructions in English alphabets but in coded form.
- ✓ These coded English words are called **mnemonics**.
- ✓ This language is called assembly language.
- ✓ Assembler is a program which converts the assembly language into machine language.
- ✓ Assembler directives are the pseudo instructions for the assembler. These tell the assembler how to convert the assembly language in binary language.

Assembly Programming Language

- ✓ An assembler reads a single assembly language source file and produces an object file.
- ✓ The object file is made up of machine instructions.
- ✓ A program may consist of many modules or source files. All these modules are converted to object files independently by the assembler.
- ✓ Then these independent object files are combined together along with the program library with the help of a linker.
- ✓ The linker then generates an executable file. This executable file is finally executed by the processor.



Building up a program.

Assembly Programming Language

- ✓ An assembly language program is a sequence of statements. There are three classes of statements:
  - ✓ Instructions - Instructions represent a single machine instruction in symbolic form.
  - ✓ Pseudo-operations - Pseudo-operations cause the assembler to initialize or reserve one or more words of storage for data, rather than machine instructions.
  - ✓ Directives - Directives communicate information about the program to the assembler, but do not generally cause the assembler to output any machine instructions.

Assembly Programming Language

- ✓ An assembly statement contains four fields:
  - ✓ **Label -** The label field is used to associate a symbolic address with an instruction or data location, or to define a symbolic constant using the .EQU, .REG, or .MACRO directives.
  - ✓ **Opcode -** The opcode field contains either a mnemonic machine instruction, or a pseudo-operation code, or the name of an assembler directive.
  - ✓ **Operands -** The operands field follows the opcode field, separated by a blank or tab. Operands are separated by commas. The meaning of the operands depends on the specific statement type, determined by the opcode.
  - ✓ **Comments -** The comments field is introduced with a semicolon, and causes the Assembler to ignore the remainder of the source line. Also tells about instruction what to perform.

Assembly Programming Language

- ✓ **Editor :** Editor is a program which is used to edit, compile and debug a file containing the assembly language program. When the program is typed, it is stored in the memory. This file is called source file.
- ✓ **Assembler :** An assembler program is used to translate the assembly language mnemonics for instructions to the corresponding machine codes. So an assembler translates a file of assembly language statements into a file of binary machine instructions and binary data.
- ✓ **Linker :** Linker is a program which is used to combine numerous object files into one object file and convert this object file into an executable file. The linker generates a link file, which contains the machine codes for all the combined sections of assembly programs.
- ✓ **Loader :** Loader is a program, which assigns absolute addresses to the program. These addresses are generated, by adding to all the offsets, the address from where the program is loaded into the memory. Loader comes into action, when you execute your program. This program is brought from the secondary memory, like disk, into the main memory at a specific address.
- ✓ **Debugger :** Debugger is a program which loads the object code program into system memory. It also troubleshoots, debug and execute the program. During and after executing the program, it monitors the contents of registers and memory locations. We can also put a breakpoint for debugging the program. If a breakpoint is inserted in a program, the debugger will run the program up to the instruction where the breakpoint is set and stop execution and display the result up to that point so that the user can check its program.

ASSEMBLER DIRECTIVES- Data

- ✓ Assembler directives are the direction (what to do and in which way) or the instructions to the assembler rather than the processor. These are also called pseudo instructions.
- ✓ Assembler directives tell the assembler how to translate a program in machine codes.
- ✓ These directives may be classified as data defining directives, segment defining directives, segment combining directives, processor directives, etc.

Data Defining Assembler Directives : These directives are used to define the type of data stored in the memory. These directives are DB, DW, DD, DQ and DT.

- ✓ **DB (Define byte):** The define byte directive is used to allocate and initializes one or more bytes of data. Here name is the symbol assigned to the variable which represents the address of the memory where the data is stored in a particular segment. The Syntax of this directive is
  Name DB data 1, data 2, data 3... For example, MEM DB 35H, 0FH, 6DH

- ✓ **DW (Define word):** This directive is used to allocate or initialize one or more data in word (16-bit) format. The syntax of this directive is Name DW word 1, word 2, word 3...
  MEM DW 0F35H, 456DH

ASSEMBLER DIRECTIVES- Data

✓ **DD (Define double word) :** This directive is used to allocate and initialize one or more data in double words (4 bytes) format. This directive is used to show that the data stored in memory is a double word. The format of this directive is: name DD Double word 1, Double word 2...

✓ **DQ (Define quad words) :** DQ directive is used to allocate and initialize one or more quad words (8 bytes) of data. The syntax of DQ is name DQ Quad word 1, Quad word 2,...

✓ **DT (Define ten bytes) :** This directive allocates and optionally initializes 10 bytes of storage for each initialize. The syntax of this directive is name DT initializer, initializer... This directive is very important while writing programs involving a math coprocessor. The data registers in math coprocessor 8087 are of 80 bits and hence data stored in the coprocessor is of 80 bits or ten bytes long.

✓ **! (Uninitialized value):** It is used as an initializer in data declarations. It indicates a value that the assembler allocates but does not initialize. The syntax of this directive is **!** . It can be used alone or with DUP, as in the following examples: Mem DB 20 DUP (**!**) ; Allocate 20 uninitialized bytes.

✓ **PTR (Pointer):** The PTR (Pointer) directive is used to define the size of an operand or the distance a reference has. The syntax of this directive is: Type PTR expression. For example: WORD PTR [BX];

✓ **OFFSET**: The OFFSET directive is used to load the offset of memory location to a pointer register.

```
Sunil Segment
Num1 DW F012H
Num2 DB A0H
Sunil Ends
```

Now if we use the instruction in the code segment MOV SI, offset num 1, it means that the offset of num 1 is loaded in SI register. Whereas MOV SI, num 1, means F012H is loaded in SI.

ASSEMBLER DIRECTIVES- Data

✓ **EQU (Equate):** This directive is used to assign a constant or a variable or an expression to a variable. Once we allot a value to a variable with the help of EQU directive, then that variable will be replaced by the assigned value throughout the program. The syntax of the EQU directive is: Variable EQU: Variable or constant or expression; for example: Pi EQU 3.14

✓ **DUP (Duplicate):** The DUP directive can be used to initialize several locations and to assign values to these locations. The DUP directive will take the following format: Name Data Type Num DUP (value). For example, Table DW 10 DUP (5). Here assembler reserves an array of 10 words of memory and initializes all 10 words with 5 and the array name is TABLE.

✓ **LABEL:** It creates a new variable or label of a given size (type) at a specified location by assigning the current location-counter value and the given type to name. It can be used to define a second entry point into a procedure. The syntax of this directive is: name LABEL type; Name is the symbol assigned to the label and type may be any one of BYTE, WORD, DWORD, QWORD, TBYTE, NEAR, FAR, PROC or a previously defined structure.

✓ **ORG:** ORG directive is used to begin an assembly program at a specific offset address. This directive is very useful which must begin assembly at 100H. The syntax of this directive is: ORG expression; For example, ORG 2000H will start a program from an offset 2000H.

✓ **EVEN:** The EVEN directive aligns next variable or instruction on an even offset address. suppose there are large amount of words stored in memory with an odd starting address, then we can use the EVEN directive to align all the words to start with an even address.

ASSEMBLER DIRECTIVES- Segment

These directives are used to define a memory segment. Segments are defined with the segment and ends directives. We can put as many segments as we like in your program.

✓ **SEGMENT:** It defines a program segment called name (any arbitrary name data, sumit or any other name). The statements are the body of the program or data within that segment. Foe example:

```
name SEGMENT
statements
```

✓ **ENDS:** The ENDS directive marks the end of a segment name or a structure name previously begun with SEGMENT. The syntax of this directive is: Name ENDS

For example, SUMIT SEGMENT

```
NUM1 DB 0F0H
RES1 DB 00
RES2 DB 00
SUNIL ENDS
```

✓ **END:** The END directive marks the end of a module and optionally indicates the address where execution will begin when the program is loaded. Only one module in a program can define a start_address. The syntax of this directive is: END [start address]

ASSEMBLER DIRECTIVES- Segment

✓ **ASSUME:** ASSUME directive is used to tell the assembler, which segment is to be used as an active segment at any instant, and with respect to which it has to calculate the offsets of the variables or instructions. You can have as many additional ASSUMES as you like. Each time an ASSUME is encountered, the assembler starts to calculate the offset with respect to that segment. The ASSUME directive is used to tell the assembler that the name given to a logical segment actually belongs to which microprocessor segment. The syntax of the ASSUME directive is:

ASSUME segment register: name

The assembler uses this information to calculate offsets correctly. Each time we load a new value into a segment register, we should inform the assembler of this fact by using an ASSUME statement. For example, consider

```
SUMIT SEGMENT
      NUM1 DB 0F0H
      NUM2 DB 0A0H
      RES1 DB 00
SUNIL ENDS
```

Here if we say ASSUME DS: SUMIT, then assembler will consider the segment whose name is SUMIT as the data segment. And if we say ASSUME ES: SUMIT, then the assembler will consider the SUMIT segment as an extra segment.

ASSEMBLER DIRECTIVES-
Initialization of Program Memory Models

✓ **.MODEL:** This directive initializes the program memory model. It should be placed at the beginning of the source file before any other simplified segment directive. .MODEL can only occur once. The syntax of the MODEL directive is: .MODEL

✓ **The MODEL directive is of five types. These are:**

- ✓ **SMALL MODEL (.MODEL SMALL) :** In this model the maximum size of the code and data segment are of 64 KB. This model is the most widely used memory model and is sufficient for all the programs.
- ✓ **MEDIUM MODEL (.MODEL MEDIUM) :** In this model the size of the data segment is maximum of 64 KB whereas the code segment size can exceed 64 KB.
- ✓ **COMPACT MODEL (.MODEL COMPACT) :** In this model the size of the code segment is maximum of 64 KB whereas the data segment size can exceed 64 KB.
- ✓ **LARGE MODEL (.MODEL LARGE) :** In this model both code and data segments size can exceed 64 KB. But it is noted that a single data set (i.e. array) can still not exceed 64 KB.
- ✓ **HUGE MODEL, (.MODEL HUGE) :** In this model both code and data segments size are more than 64 KB. Furthermore, in this model a single data set can be more than 64 KB.

ASSEMBLER DIRECTIVES- Segment

✓ **.CODE:** This directive indicates the start of a code segment and ends previous segment, if any. Before using the .CODE, the .MODEL directive must have previously been used. Each .CODE directive generates an ASSUME statement associating CS with the current segment. Syntax: .CODE [name]

✓ **.STACK:** The .STACK defines the program stack segment and ends previous segment, if any. Before using the .STACK, the .MODEL directive must have previously been used. No data and code should be placed in this segment; the processor uses the stack segment to store data during procedure and interrupt calls. The syntax of the .STACK is: .STACK [size], Here size specifies size of the stack in bytes and it is optional. If none is given, the default is 1024.

✓ **.DATA:** The .DATA starts the initialized data segment and ends previous segment, if any. Before using the .DATA, the .MODEL directive must have previously been used. This segment contains all global data that have initial values. The .MODEL directive generates a GROUP statement that places _DATA in DGROUP.

✓ **.DATA!:** This directive starts the uninitialized data segment and ends previous segment, if any. Before using the .DATA?, the .MODEL directive must have previously been used. This segment contains all global data that have no initial values. The syntax of this directive is: .DATA?

ASSEMBLER DIRECTIVES-
Initialization of Program Memory Models

✓ **.STARTUP:** This directive is used to generate startup code for the given model and stack type which is defined by the .MODEL directive. It initializes data segment (DS), stack segment (SS) and stack pointer (SP) as and when required. It defines a start- address label, so that you don't need to give a start address with END. It is used in stand-alone programs only. The syntax of STARTUP directive is: .STARTUP

The program which is started with the .STARTUP directive is always ended with the .EXIT directive at the end of the program.

✓ **.EXIT:** The .EXIT directive is used to end a program which is initiated with the .STARTUP directive. In fact this directive generates the type 21H interrupt, to exit from the program. This directive is used only when the .MODEL directive has been used earlier and a .STARTUP directive must have been used in that model. The syntax of this directive is: .EXIT [exitcode]

ASSEMBLER DIRECTIVES

PROCEDURE	MACRO
❖ To use PROCEDURE, we have to use CALL instruction.	❖ To use MACRO, we have to use LABEL only.
❖ By PROCEDURE, we perform subroutine.	❖ By MACRO, We insert function in code.
❖ PROCEDURE acquires size only once.	❖ MACRO may acquires size many times in program.
❖ PROCEDURE is based on CALL. So, it uses the STACK for PUSH and POP of subroutine.	❖ MACRO is function. We can use function with MACRO which is not using STACK.
❖ PROCEDURE executes branch.	❖ MACRO doesn't executes branch.
❖ It affects the pipelining as it is branch.	❖ It does not affects the pipelining.
❖ It is slower, but requires less memory space.	❖ It is Faster, but may needs more memory space.

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to add two 8-bit numbers.

Solution : First the data segment is defined by segment defining statement. In the data segment the two numbers are entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive.

The memory location which is defined by the name Data is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot be loaded with the immediate data. One number is loaded into AX register and then the second number is added with the contents of AX. The result is placed in the memory location named 'result'.

```
DATA1 SEGMENT
       NUM1 DB 85H              : Number1 is declared as word
       NUM2 DB 34H              : Number2 is declared as word
       RESULT DB 00             : RESULT is initialized
DATA1 ENDS
CODE SEGMENT                    : Code segment is defined
       ASSUME CS: CODE, DS:DATA1  : Labels of the code and data segment are
                                    assigned to CS & DS
START:                          : Start of the program code
       MOV AX, DATA1            : The memory location which is defined by
                                    the  name data is
       MOV DS, AX               : Loaded into DS through AX register
       MOV AX, 00H              : AX is cleared
       MOV AL, NUM1             : Number1 is moved to the AX register
       ADD AL, NUM2             : Number2 is added with the contents of the
                                    AX register
       MOV RESULT, AL           : Contents of AX register is moved to the
                                    RESULT
       MOV AX, 4C00H
       INT 21H                  : Stop the program execution
CODE ENDS                       : Code segment ends
END START                       : Program code ends
END
```

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to add two 16-bit numbers.

Solution : First the data segment is defined by segment defining statement. In the data segment the two numbers are entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive.

The memory location which is defined by the name Data is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot be loaded with the immediate data. One number is loaded into AX register and then the second number is added with the contents of AX. The result is placed in the memory location named 'result'.

```
DATA SEGMENT
        NUM1 DW 6785H              : Number1 is declared as word
        NUM2 DW 1234H              : Number2 is declared as word
        RESULT DW 00               : RESULT is initialized
DATA ENDS
CODE SEGMENT                       : Code segment is defined
        ASSUME CS: CODE, DS:DATA   : Labels of the code and data segment are
                                     assigned to CS & DS
START:                             : Start of the program code
        MOV AX, DATA               : The memory location which is defined by
                                     the  name data is
        MOV DS, AX                 : Loaded into DS through AX register
        MOV AX, 00H                : AX is cleared
        MOV AX, NUM1               : Number1 is moved to the AX register
        ADD AX, NUM2               : Number2 is added with the contents of the
                                     AX register
        MOV RESULT, AX             : Contents of AX register is moved to the
                                     RESULT
        MOV AX,4C00H
        INT 21H                    : Stop the program execution
CODE ENDS                          : Code segment ends
END START                          : Program code ends
END
```

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to subtract two 8-bit numbers.

Solution : First the data segment is defined by segment defining statement. In the data segment the two numbers are entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive.

The memory location which is defined by the name Data is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot be loaded with the immediate data.

One number is loaded into AL register and then the second number is loaded into BL register, and subtraction is performed between them. The result is placed in the memory location named 'result'

```
DATA SEGMENT                    : Data segment is defined by segment defining
                                  statement

        NUM1 DB 9FH             : Number1 is declared as byte
        NUM2 DB SAH             : Number2 is declared as byte
        RESULT DB 00            : RESULT is initialized
DATA ENDS
CODE SEGMENT                    : Code segment is defined
        ASSUME CS: CODE, DS: DATA
START:                          : Start of the program code
        MOV AX, DATA            : The memory location which is defined by
                                  the name data is
        MOV DS, AX              : Loaded into DS through AX register
        MOV AX, 00              : AX is cleared
        MOV AL, NUM1            : Number1 is moved to the AL register
        MOV BL, NUM2            : Number2 is moved to the BL register
        SUB AL, BL              : Subtraction between the two numbers is
                                  performed
        MOV RESULT, AL          : Move subtraction result to 'RESULT'
        INT 03H
CODE ENDS                       : Code segment ends
END START                       : Program code ends
END
```

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to subtract two 16-bit numbers.

Solution : First the data segment is defined by segment defining statement. In the data segment the two numbers are entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive.

The memory location which is defined by the name Data is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot be loaded with the immediate data.

One number is loaded into AL register and then the second number is loaded into BL register, and subtraction is performed between them. The result is placed in the memory location named 'result'

```
DATA1 SEGMENT
        NUM1 DW 0F0F0H          : Number1 is declared as word
        NUM2 DW 0ABCDH          : Number2 is declared as word
        RESULT DW 00            : RESULT is initialized
DATA1 ENDS
CODE SEGMENT                    : Code segment is defined
        ASSUME CS: CODE, DS: DATA1
START:                          : Start of the program code
        MOV AX,DATA1            : The memory location which is defined by
                                  the name data is
        MOV DS,AX               : Loaded into DS through AX register
        MOV AX,00               : AX is cleared
        MOV AX,NUM1             : Number1 is moved to the AX register
        MOV BX,NUM2             : Number2 is moved to the BX register
        SUB AX,BX               : Subtraction between these two numbers is
                                  performed
        MOV RESULT,AX           : Move subtraction result to 'RESULT'
        INT 03H
CODE ENDS                       : Code segment ends
END START                       : Program code ends
END
```

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to multiply two 8-bit numbers.

Solution : First the data segment is defined by segment defining statement. In the data segment the two numbers are entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive.

The memory location which is defined by the name SUNIL is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot be loaded with the immediate data.

One number is loaded into AL register and the second number is loaded into BL register, then multiplication between these two numbers is performed. The lower order byte of the result is placed in the memory location named 'RES1' and the higher order byte is placed in the memory location named 'RES2'.

```
SUNIL SEGMENT
        NUM1 DB 0F0H                : Number1 is declared as byte
        NUM2 DB 0A0H                : Number2 is declared as byte
        RES1 DB 00                  : RES1 is initialized for lower order
                                      byte of result
        RES2 DB 00                  : RES2 is initialized for higher order
                                      byte of result
SUNIL ENDS
MATHUR SEGMENT                      : Code segment is defined
        ASSUME CS: MATHUR, DS: SUNIL
START:
        MOV AX, SUNIL               : The memory location which is defined
                                      by the name sunil is
        MOV DS, AX                  : Loaded into DS through AX register
        MOV AX, 00                  : AX is cleared
        MOV AL, NUM1                : Number1 is moved to the AL register
        MOV BL, NUM2                : Number2 is moved to the BL register
        MUL BL                      : Multiply no. 1 with no. 2
        MOV RES1, AL                : Move lower byte of result in RES1
        MOV RES2, AH                : Move higher byte of result in RES2
        INT 03H                     : Stop the program execution
MATHUR ENDS                         : Code segment ends
END START                           : Program code ends
END
```

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to perform 16-bit division.

Solution : First the data segment is defined by segment defining statement. In the data segment the two numbers are entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive.

The memory location which is defined by the name Data is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot be loaded with the immediate data.

One number is loaded into AX register and the second number is loaded into BX register, then division between these two numbers is performed. After division the content of AX is placed into the 'QUOTIENT' and the contents of DX is placed into the 'REMAINDER'.

```
DATA SEGMENT
        DATA1 DW 0F0F0H      : Data1 is declared as word
        DATA2 DW 0A0A0H      : Data2 is declared as word
        QUOTIENT DW 00       : Quotient of the result is initialized
        REMAINDER DW 00      : Remainder of the result is initialized
CODE SEGMENT                 : Code segment is defined
ASSUME CS: CODE, DS: DATA    : Labels of the code and data segment are
                               assigned
START:                       : Program code start
        MOV AX,DATA          : The memory location which is defined by the
                               name data is
        MOV DS,AX            : Loaded into DS through AX register
        MOV AX,00            : AX is cleared
        MOV DX,00            : DX is cleared
        MOV AX,DATA1         : Move data1 to AX register
        MOV BX,DATA2         : Move data 2 to BX register
        DIV BX               : Divide data1 by data2
        MOV QUOTIENT,AX      : Move the content of AX register to QUOTIENT
        MOV REMAINDER,DX     : Move the content of DX register to REMAINDER
        INT 03               : Stop the program execution
CODE ENDS                    : Code segment ends
END START                    : Program code ends
END
```

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to find the largest in a series.

Solution : First the data segment is defined by segment defining statement. In the data segment one number is entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive.

The memory location which is defined by the name Data is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot be loaded with the immediate data.

```
DATA SEGMENT
NUM DB 06H, 02H, 86H, 08H, 04H, 76H, 07H
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE, DS: DATA

START:

MOV AX,DATA ; The memory location which is defined by the name data is
MOV DS,AX   ; Loaded into DS through AX register
LEA SI,NUM  ; Effective address of the NUM1 is transferred to SI
XOR CH,CH   ; Clear Ch register by exclusive-OR operation
MOV CL,[SI] ; Initialize CL register as counter
DEC CL  ; Decrement counter by1
INC SI  ; Increment SI register by1
MOV AL,[SI] ; Move the content of [SI] to AL register
UP2: CMP AL,[SI+1]   ; Compare the content of AL with the contentof [SI+1]
 JNC UP1    ; jump if first operand is above or equal to second operand
MOV AL,[SI+1]    ; Move the content of [SI+1] to AL register
UP1:    INC SI  ; Increment the content of SI by 1
DEC CL
JNZ UP2 ; If counter is not o, go to UP2
MOV [SI+1],AL   ; Move the content of AL register to [SI+1]
hlt ; stop the execution
CODE ENDS
END START
END
```

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to count a string length.

Solution : First the data segment is defined by segment defining statement. In the data segment the one strings is entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive.

The memory location which is defined by the name Data is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot be loaded with the immediate data.

```
DATA1 SEGMENT
     STR DB 'SUMITKUMARGUPTA'
     LEN EQU $-STR
DATA1 ENDS

CODE1 SEGMENT
     ASSUME CS:CODE1 DS:DATA1

START:
     MOV AX,DATA1
     MOV DS,AX
     XOR AX,AX
     MOV AX,LEN
     HLT
CODE1 ENDS
END START
END
```

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to arrange a string in reverse order

Solution : First the data segment is defined by segment defining statement. In the data segment the two strings are entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive.

The memory location which is defined by the name Data is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot be loaded with the immediate data.

```
DATA SEGMENT
STRING1 DB 'MITWPU'
STRING2 DB 12 DUP(0)
DATA ENDS
CODE SEGMENT
ASSUME CS: CODE,DS: DATA,ES: DATA
START: MOV AX,DATA
MOV DS,AX
MOV ES,AX
MOV BX,OFFSET STRING1 ; Initialize BX with the offset of STRING1
MOV SI,BX   ; Move the content of BX register to SI
MOV DI,OFFSET STRING2 ; Initialize DI with the offset of STRING2
ADD DI,0CH  ; Add total number of string data to DI
CLD ; Clear the direction flag
MOV CX,0CH  ; Set the counter by total number of string data

UP:   MOV AL,[SI]   ; Move the content of [SI] to AL register
MOV ES: [DI],AL   ; Move the content of AL register to [DI]
INC SI    ; Increment the content of SI by 1
DEC DI  ; Decrement the content of DI by 1
LOOP UP ; Go to UP if CX is not 0
REP MOVSB   ; MOVSB is repeated until CX is zero
HLT
CODE ENDS
END START
END
```

ALP : Write a program to concatenate two string data.

```
; Program to concat two string
.MODEL SMALL
.STACK 100H
.DATA
 STR1 DB 'sumit$'
 STR2 DB 'Gupta$'
 L1 DB 0
 L2 DB 0
.CODE
 MOV AX , @DATA  ; Initializing Data Segment
 MOV DS , AX   ; Finding Length of STR1
 MOV DI , OFFSET STR1

  UP1:
  MOV AL , [DI]
  CMP AL , '$'
  JE DN1
   INC DI
   INC L1
  JMP UP1
 DN1:    ; Concating STR1 and STR2
 LEA SI , STR2  ; Loading Effective Address i.e Base Address of STR2
   UP:
   MOV  AL  , [SI]
   MOV [DI] , AL
   CMP AL , '$'
   JE DN
   INC SI
   INC DI
   JMP UP

DN:    ; Printing String After Concatination
 MOV AH , 09H
 LEA DX , STR1
 INT 21H
 MOV AH , 4CH  ; Service routine for exit
 INT 21H
END
```

ALP : Write a program to check whether string is palindrome or not.

```
.MODEL SMALL
.STACK 100H
.DATA
     STRING DB 'abccba', '$'
     STRING1 DB 'String is palindrome', '$'
     STRING2 DB 'String is not palindrome', '$'
.CODE
MAIN PROC FAR
     MOV AX, @DATA
     MOV DS, AX
     CALL Palindrome
     MOV AH, 4CH
     INT 21H
MAIN ENDP
 Palindrome PROC
      MOV SI, OFFSET STRING
LOOP1:
     MOV AX, [SI]
     CMP AL, '$'
     JE LABEL1
     INC SI
     JMP LOOP1
LABEL1:
     MOV DI, OFFSET STRING
     DEC SI
 LOOP2:
     CMP SI, DI
     JL OUTPUT1
     MOV AX, [SI]
     MOV BX, [DI]
     CMP AL, BL
     JNE OUTPUT2
     DEC SI
     INC DI
     JMP LOOP2
OUTPUT1:
     LEA DX, STRING1
     MOV AH, 09H
     INT 21H
     RET
OUTPUT2:
     LEA DX, STRING2
     MOV AH, 09H
     INT 21H
     RET
Palindrome ENDP
END MAIN
```

ASSEMBLER DIRECTIVES – Examples

ALP : Write a program to compare two string data.

Solution : First the data segment is defined by segment defining statement. In the data segment the three strings and one string length are entered. Then code segment is defined and the labels of the code and data segments are assigned to the CS and DS segments by assume directive. The memory location which is defined by the name Data is loaded into the DS segment register through AX register. The AX register here is used because segment registers cannot
be loaded with the immediate data.

ASSEMBLER DIRECTIVES – Examples

ALP : Assembly program on Addition, Subtraction, Multiplication and Division

```
MOV AX, [1100H]; Move content of 1100H into Ax register.
MOV BX, [1102H]; move content of 1102H into BX
ADD AX, BX ; Addition of Ax and BX.
MOV [1110H], Ax; Move Ax into content of 1110.
MOV AX,[1100H]; Move 100 content of 1110m into Ax
SUB AX, BX ; Substract Ax and BX.
MOV [1120H] AX ; Move the content Ax into content of 1120.
MOV AX,[1100H]; Move 100 content of 1110m into Ax
MUL BX; Multiplication of BX
MOV [1130H], AX; Move content Ax into content od 1130H.
Mov [1132H],Dx; Move DX into content of 1132H.
MOV AX, [1100h]; Move content of 1100m into AX.
DIV BX ; Division of BX with AX.
MOV [1140H], AX; Move Ax into content of 1140H
MOV [1142H], DX; Move Dx into content of 1142H
HLT.
```

ASSEMBLER DIRECTIVES – Examples

ALP : Sum of Series

```
MOU AX, [1100H];Move content of 1100H into Ax
MOU BX, [1200H];Move content of 1200H into Bx
MOU CX, [1300H];Move content of 1300H into Cx
REPEAT:
ADD AX, BX;Addition Ax and Bx
INC [1200H];Incrementation in content of 1200H.
DEC CX;Decrementation in CX
JNZ REPEAT;Jump on REPEAT when zero flag is not set
MOU [1400H], AX; Move AX into content of 1400h
HLT
```

ASSEMBLER DIRECTIVES – Examples

ALP : Smallest numbers from array

```
MOV SI, 1100H; Move memory location 1100H into SI
Mov DI, 1200H; Move memory location 1200H into DI
MOV CL, [SI]; Move content of SI into CL.
INC SI ;Incrementation in SI
MOV AL, [SI]; Move content of SI into AL.
DEC CL; Decrementation in el.
Again:
INC S1 ;Incrementation in SI
MOV BL, [SI]; move content of SI in to BL.
CMP AL, BL; compair AL and BL.
JC Ahead; Jump on ahead when carry set.
MOV AL, BL; Move BL into AL.
Ahead:
DEC CL; Decrementation in CL.
JNZ Again; Jump on again when zero flag is not set
MOV [DI], AL; Move Al into content of DI.
HLT
```

ASSEMBLER DIRECTIVES – Examples

ALP : Largest numbers from array

```
Mov SI, 1100H;Move the memory location 1100H into SI
MOV DI, 1200H;Move the memory location 12000 into DI
MOV CL, [SI] ;Move the content of [SI] into CL
Inc SI ;Increment in SI
MOV AL, [SI]; Move content of SI into AL
DEC CL; Decrement in CL
AGAIN:
INC SI;Increment in SI
MOVE BL,[SI];Move the content of SI into BL
CMP AL,BL ;Compare AL and BL
JNC AHEAD; Jump ahead if carry flag is not set on AHEAD
MOV AL, BL ;Move BL into AL
AHEAD:
DEC CL; decrement in CL
JNZ AGAIN; Jump if zero flag is not set on AGAIN
MOV [DI], AL ;Move Al into DI content.
HLT
```

ASSEMBLER DIRECTIVES – Examples

ALP : Sorting numbers in Descending order

```
MOV SI, 1100H;Move memory content of 1100h to SI
MOV CL, [SI];Move content of SI into CL.
DEC CL;Decrementation in CL
REPEAT:
MOV SI, 1100H;Move memory location lloon into SI
MOV CH,[SI];Move content of SI into CH.
DEC CH;decrementation in CH.
INC SI; Incrementation in SI.
REPCOM:
MOV AL, [SI];move content of si into AL
INC SI;Incrementation in SI
CMP AL, [SI];Compair Al and content of SI
JNC AHEAD;Jump on ahead when carry flag is not set
XCHG AL, [SI];Exchange AL with content of SI
XCHG AL, [SI-1]; Exchange Al with content of SI-1
AHEAD:
DEC CH;Decrementation in CH.
JC REPCOM;Jmp if carry flag is set to 1 on Repcom
DEC CL;Decrementation in  CL
JNZ REPEAT;Jump on REPEAT when zero flag is not set
HLT
```

ASSEMBLER DIRECTIVES – Examples

ALP : Sorting numbers in Ascending order

```
MOU SI, 1100H;Move memory content of 1100h to SI
MOU CL, [SI];Move content of SI into CL.
DEC CL;Decrementation in CL
REPEAT:
MOU SI, 1100H;Move memory location lloon into SI
MOU CH,[SI];Move content of SI into CH.
DEC CH;decrementation in CH.
INC SI; Incrementation in SI.
REPCOM:
MOU AL, [SI];move content of si into AL
INC SI;Incrementation in SI
CMP AL, [SI];Compair Al and content of SI
JNC AHEAD;Jump on ahead when carry flag is not set
XCHG AL, [SI];Exchange AL with content of SI
XCHG AL, [SI-1]; Exchange Al with content of SI-1
AHEAD:
DEC CH;Decrementation in CH.
JNZ REPCOM;Jump on REPCOM when zero flag is not set
DEC CL;Decrementation in  CL
JNZ REPEAT;Jump on REPEAT when zero flag is not set
HLT
```

ASSEMBLER DIRECTIVES – Examples

ALP : Block transfer

```
MOV SI,1100H ; COPY OF MEMORY LOCATION FOR SOURCE DATA
MOV DI,1200H ; COPY OF DESTINATION FOR DATA TRANSFER
MOV CL,[SI] ; COPY THE NUMBER OF DATA TO BE TRANSFER FOR COUNTER
INC SI  ; INCREMENT THE MEMORY LOCATION FOR DATA BLOCK
REP MOVSB  ; REPEAT THE COPY INSTRUCTION TILL COUNTER IS ZERO
HLT  ; STOP THE EXECUTION
```

```
MOV SI,1100H ; COPY OF MEMORY LOCATION FOR SOURCE DATA
MOV DI,1200H ; COPY OF DESTINATION FOR DATA TRANSFER
MOV CL,[SI] ; COPY THE NUMBER OF DATA TO BE TRANSFER FOR COUNTER
INC SI  ; INCREMENT THE MEMORY LOCATION FOR DATA BLOCK
NEXT:   ; LEVEL DEFINE
MOV AL,[SI]   ; COPY SOURCE DATA TO AL REGISTER
MOV [DI],AL   ; COPY AL REGISTER DATA TO DESTINATION REGISTER
INC SI      ; INCREMENT SOURCE INDEX
INC DI      ; INCREMENT DESTINATION INDEX
DEC CL      ; DECREMENT THE COUNTER
JNZ NEXT  ; CHECK THE COUNTER IS ZERO OR NOT USING ZERO FLAG
HLT  ; STOP THE EXECUTION
```

ASSEMBLER DIRECTIVES – Examples

ALP : String Operations - Length, Reverse, Compare, Concatenation, Copy